Міністерство освіти і науки України
Національний технічний університет України
"Київський політехнічний інститут імені Ігоря Сікорського"
Фізико-технічний інститут

Криптографія
Комп’ютерний практикум №3

Виконали студенти гр. ФБ-42

Бобров Павло

Ходячих Нікіта

Київ – 2026

Комп’ютерний практикум №.3

Мета роботи:

Набуття навичок частотного аналізу на прикладі розкриття моноалфавітної підстановки; опанування прийомами роботи в модулярній арифметиці.

Порядок виконання роботи

0. Уважно прочитати методичні вказівки до виконання комп’ютерного практикуму.
1. Реалізувати підпрограми із необхідними математичними операціями: обчисленням оберненого елементу за модулем із використанням розширеного алгоритму Евкліда, розв’язуванням лінійних порівнянь. При розв’язуванні порівнянь потрібно коректно обробляти випадок із декількома розв’язками, повертаючи їх усі.
2. За допомогою програми обчислення частот біграм, яка написана в ході виконання комп’ютерного практикуму №1, знайти 5 найчастіших біграм запропонованого шифртексту (за варіантом).
3. Перебрати можливі варіанти співставлення частих біграм мови та частих біграм шифртексту (розглядаючи пари біграм із п’яти найчастіших). Для кожного співставлення знайти можливі кандидати на ключ (a,b) шляхом розв’язання системи (1).
4. Для кожного кандидата на ключ дешифрувати шифртекст. Якщо шифртекст не є змістовним текстом російською мовою, відкинути цього кандидата.
5. Повторювати дії 3-4 доти, доки дешифрований текст не буде змістовним

Хід роботи

Спочатку візьмемо Тестовий варіант ШТ для налаштування і тестів всього коду. Напишемо базову логіку відповідно до формул та гляну на результат першого комп’ютерного практикуму щоб виписати найпопулярніші символи та біграми російської мови.

Трохи обріжемо код та виведу n-грами для зручного копіювання:

```
================================================
 РЕЗУЛЬТАТИ ДЛЯ: cleaned_war_nospaces.txt
================================================
Довжина тексту: 1485071 символів

--- Монограми (H1) ---
H1 = 4.466633 біт/символ
Топ-5 найчастіших символів:
'о', 'е', 'а', 'и', 'н', 'т', 'с', 'л', 'в', 'р', 'к', 'д', 'м', 'у', 'п',
--- Біграми, що не перетинаються (H2) ---
H2 (не перетинаються) = 4.147440 біт/символ
Топ-5 найчастіших біграм:
'то', 'ст', 'на', 'ов', 'ал', 'го', 'он', 'не', 'но', 'ко', 'ен', 'по', 'ос', 'ра', 'от',
```

Використємо ці 20 найпопулярніших біграм та 10 найпопулярніших символів

*в завданні вказано 5 біграм, але за такої кількості на тестовому тексті не знаходився ключ, а більша кількість підвищила точність

Пишемо код:

Перші 45 строчок – імпорти та математичні функції
46-54 – перевірка щоб у розшифрованому тексті не було синтаксично неможливих біграм та щоб сума частот найпопулярніших символів була >50%
55-74 – сама функція для знаходження кандидатів ключа, на основі наших найпопулярніших біграм та відкидання кандидата, якщо розшифрований ним текст є потенційно неможливим

75-79 – main

```python
from collections import Counter
from itertools import permutations
import os

DIR = os.path.dirname(os.path.abspath(__file__))
ALPHABET = "абвгдежзийклмнопрстуфхцчшщыьэюя"
```

```
M = len(ALPHABET); M2 = M * M
IDX = {c: i for i, c in enumerate(ALPHABET)}

def egcd(a, b): # алгоритм евкліда
    u0, u1, v0, v1 = 1, 0, 0, 1
    while b:
        q = a // b
        a, b = b, a - q * b
        u0, u1 = u1, u0 - q * u1
        v0, v1 = v1, v0 - q * v1
    return a, u0, v0

def inv(a, n): # обернений елемень
    g, u, _ = egcd(a % n, n)
    return u % n if g == 1 else None

def solve_lin(a, b, n): # розв лінійного рівняння
    a, b = a % n, b % n
    d, _, _ = egcd(a, n)
    if b % d:
        return []
    a1, b1, n1 = a // d, b // d, n // d
    x0 = (b1 * inv(a1, n1)) % n1 if n1 > 1 else 0
    return [x0 + k * n1 for k in range(d)]

def bigram_to_num(s):  return IDX[s[0]] * M + IDX[s[1]]           # Біграми в
числове значення та навпаки
def num_to_bigram(x):  return ALPHABET[x // M] + ALPHABET[x % M]  #

def top_bigrams(text, k=5): # найпопулярнішій біграми
    bg = [text[i:i+2] for i in range(0, len(text) - 1, 2)]
    return [b for b, _ in Counter(bg).most_common(k)]

def decrypt(text, a, b):
    ai = inv(a, M2)
    out = []
    for i in range(0, len(text) - 1, 2):
        out.append(num_to_bigram((ai * (bigram_to_num(text[i:i+2]) - b)) %
M2))
    return "".join(out)

FORBIDDEN =
["аь","еь","ыь","ьь","йь","оь","уь","яь","юь","жы","шы","щы","чы","щю","чя","щ
я","фь","ьы","ьй","йы","йй","ьь"]
COMMON_CHARS = "оеаинтслвр"
def is_meaningful(t):
    if any(f in t for f in FORBIDDEN):
        return False
```

```
    match_count = sum(1 for c in t if c in COMMON_CHARS)
    return (match_count / len(t)) > 0.50

RU_TOP_BG = ['то', 'ст', 'на', 'ов', 'ал', 'го', 'он', 'не', 'но', 'ко', 'ен',
'по', 'ос', 'ра', 'от']

def attack(cipher, RU_TOP_BG=RU_TOP_BG):
    cip_top = top_bigrams(cipher, 5)
    X = [bigram_to_num(x) for x in RU_TOP_BG]
    Y = [bigram_to_num(y) for y in cip_top]
    found, seen = [], set()
    for (x1, x2) in permutations(X, 2):
        for (y1, y2) in permutations(Y, 2):
            for a in solve_lin(x1 - x2, y1 - y2, M2):
                if egcd(a, M2)[0] != 1:
                    continue
                b = (y1 - a * x1) % M2
                if (a, b) in seen: continue
                seen.add((a, b))
                pt = decrypt(cipher, a, b)
                if is_meaningful(pt):
                    found.append((a, b, pt))
    return found


cipher = open(os.path.join(DIR, "V3.txt"), encoding="utf-
8").read().replace("\n", "").strip()
print("Топ-5 біграм шифртексту:", top_bigrams(cipher))
for a, b, pt in attack(cipher):
    print(f"\nКЛЮЧ: a={a}, b={b}\n{pt[:300]}")
```

Запускаємо код і бачимо всі знайдені кандидати ключа (в данному випадку лише 1 вірний) та розшифрований текст (читабельний, тож все вірно), так само перевірено на декількох інших тестових текстах, у всіх випадках знаходився правильний ключ

```
Топ-5 біграм шифртексту: ['ыв', 'хр', 'хф', 'йз', 'ыя']

КЛЮЧ: a=890, b=102
виднарушенияявстречаетсянаиболеечастопоследствиямогутбытьсамыеразныееслипохищентексткнигис
лективаавторовэтокатастрофаипотеримогутвыражатьсявтысячахдоллароводнакоесликнигаужеиздана
```

На тексті за варіантом код не знайшов жодного кандидата, тож ми спробували позмінювати критерії відкидання кандидатів, та пошукати серед купи кандидатів правильного. Зміна частоти символів навіть до 10%

(така частота фактично взагалі нічого не відсіюватиме, а у «война и мир» загальна сума була частот цих 10 символів була 66%) не дала навіть 1 ключа, тож ми спробувати відключити фільтр за біграмами – отримали близько 20 ключів за >50% частотою (10 символів) та лише 1 ключ за >60% частотою

```
#FORBIDDEN = ["аь","еь","ыь","ьь","йь","оь","уь","яь","юь
COMMON_CHARS = "оеаинтслвр"
def is_meaningful(t):
#    if any(f in t for f in FORBIDDEN):
#        return False

    match_count = sum(1 for c in t if c in COMMON_CHARS)
    return (match_count / len(t)) > 0.60
```

```
Топ-5 біграм шифртексту: ['тд', 'рб', 'во', 'щю', 'кд']

КЛЮЧ: a=199, b=700
отцеубийствокакизвестноосновноеиизначалыноягрестнглениечеловече
йлиисследованиямнеудалосыещеустановитыдушевноепроисхыжденивинь
```

Текст вже є читабельним, проте помітна єдина постійна помилка – букви «ы» та «ь» поміняні місцями, що вірогідніше за все є саме опечаткою оригінального тексту. (помилкою дешифрування не може бути, через природу самого шифру)

Перевіривши наш код (з за коментованим Forbidden та незакоментованим) на декількох текстах інших варіантів, була виявлена така сама помилка з «ы» та «ь»

Контрольні запитання

1. Які статистичні властивості шифртексту були використані під час формування кандидатів на ключ?
-Частоти окремих символів та біграм (взяті найпопулярніші з них)

2. Чому не кожне зіставлення частих біграм відкритого тексту та шифртексту приводить до допустимого ключа?
- Можливі такі варіанти – математична необоротність ключа, відсутність розв'язку лінійного порівняння, статистичні відхилення (частоти окремих символів, абощо) та лексично неможливі біграми (напр. «ьь»)

3. Чи виникали під час криптоаналізу випадки, коли одне зіставлення біграм давало декілька кандидатів на ключ? Як такі кандидати оброблялися?
- Виникали, але перевірялися на оборотність ключа, розв'язок ЛР та тестового дешифрування з послідуючою статистичною перевіркою або перевіркою на неможливі біграми

4. За якими ознаками серед отриманих кандидатів було визначено правильний ключ?
- Оборотність ключа, статистичні ознаки природньої мови та читабельність тексту людиною

5. Який критерій автоматичного розпізнавання осмисленого тексту було використано? Чому було обрано саме його?
- Сума частот 10 найпопулярніших символів вища за 60% (+-66% для природнього тексту)
Також відсутність лексично неможливих біграм (хоч і не працювала для тексту за варіантом через помилки у ВТ)

6. Як поводився обраний критерій для правильного відкритого тексту та для результатів розшифрування неправильними ключами?
- Сума частот 10 найпопулярніших символів за значення >50% надавала 20 ключів, для >55% 4 ключі та для >60-65% 1 ключ (у всіх випадках був присутній 1 правильний). Тож що логічно у справжнього ключа ця сума частот має бути близькою 66%
Відсутність лексично неможливих біграм, коли була застосована без фільтрації за частотами, надавала 1 ключ для декількох перевірених тестових варіантів. Для великого тексту це є логічним, бо в неправильних кандидатах з високою вірогідністю хоч раз зустрінеться лексично неможлива біг рама, проте приклад з текстом за варіантом показує слабкість цього методу до помилок у ВТ.